Comparing ML Approaches for Handwritten Digit Recognition

Pakhale Kalyani Vijay, Steffen Andreas Schick, Nutchayangkul Nicha, and Gill Glenn-Robin Stokke

MSI5102 Essentials of Machine Learning | April 2026

Why Handwritten Digit Recognition Matters

Real-World Applications

- Postal mail sorting (ZIP code reading)
- Bank check processing (amount verification)
- Document digitization and OCR systems
- Tax form and invoice automation

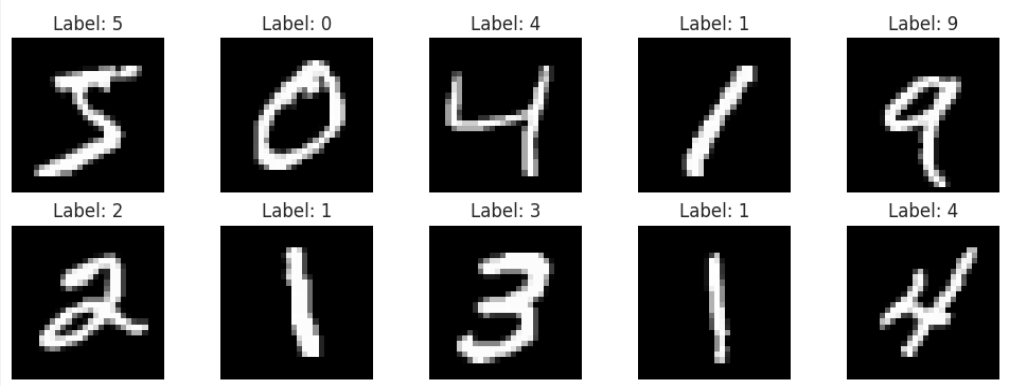
Our Research Question

- Which ML paradigm best balances accuracy, speed, and interpretability for image classification?
- We compare 7+ models across traditional ML and deep learning.
- Primary metric: accuracy and confusion matrices. Precision, recall, and F1 available in appendix.

Approach:

By representing images as **high-dimensional feature vectors**, we establish a consistent mathematical framework to compare models, starting with simple linear baselines and moving toward complex architectures to uncover the **data's underlying geometry** and **separate the digits**.

Sample MNIST Digits



MNIST Dataset: 70,000 Digits

Dataset Specifications

- **Dimensions**: 28 x 28 grayscale images (784 features)
- **Classes**: 10 roughly balanced digit classes (0–9)
- **Volume**: 60,000 training / 10,000 test samples

Data pre-processing

Normalize
/ 255 → [0, 1]

→

Reshape
784-D vector (ML)
28×28×1 (CNN)

→

Split
80% train / 20% validation
10K test held out

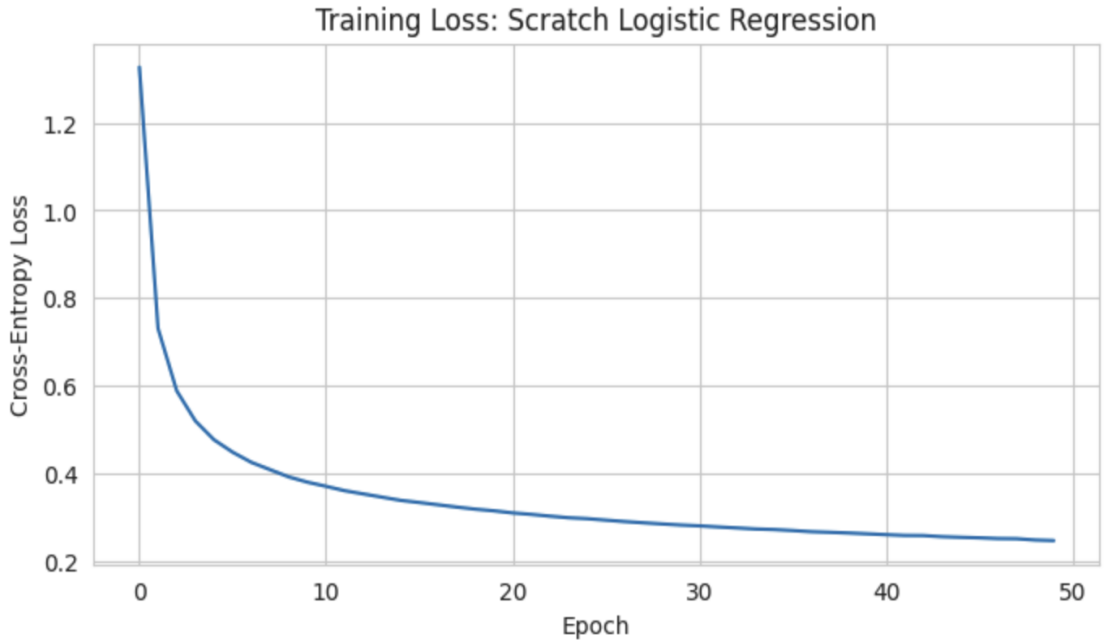
Normalizing prevents high-intensity pixels from dominating distance calculations (kNN) and ensures stable gradient convergence (Logistic Regression, CNNs).

Logistic Regression: Linear Baseline

- **Softmax:** Converts logits to probabilities.
- **Cross-Entropy Loss:** Measures prediction dissimilarity.
- **Multinomial (not OvR):** Models all 10 classes simultaneously via Softmax.
- **Convergence:** Stable after ~30 epochs due to mini-batch averaging.

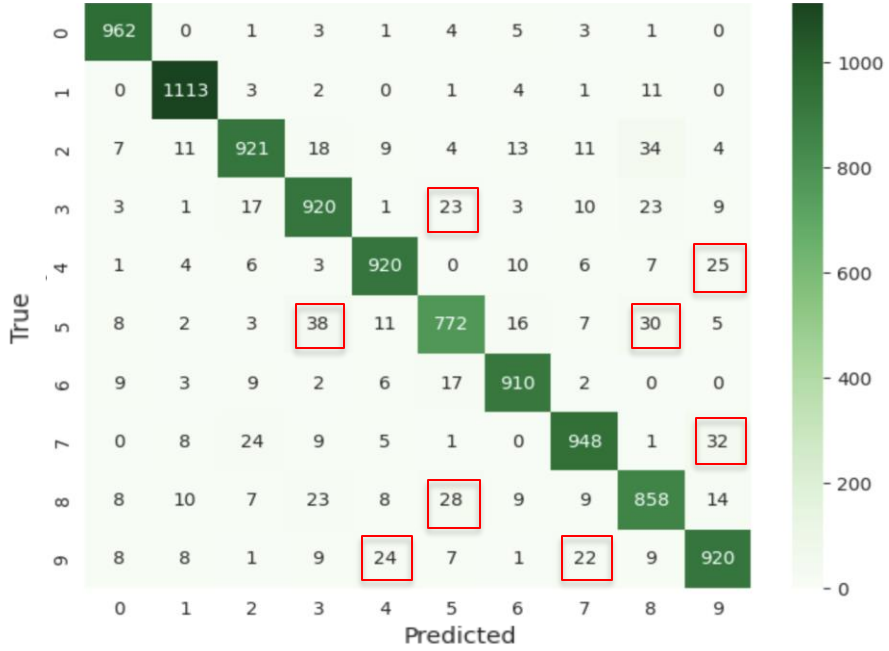
- **Scratch Model:** 90.42% accuracy.
- **Scikit-learn Model (92.44%):** Improved via full dataset training and L2 regularization to prevent overfitting.
- **Error Analysis:** Misclassifications occur between similar-looking digits (e.g., 7/9, 3/5).
- **Linear Limitation:** Cannot capture the complex non-linear boundaries needed for perfect separation.

From-Scratch Implementation

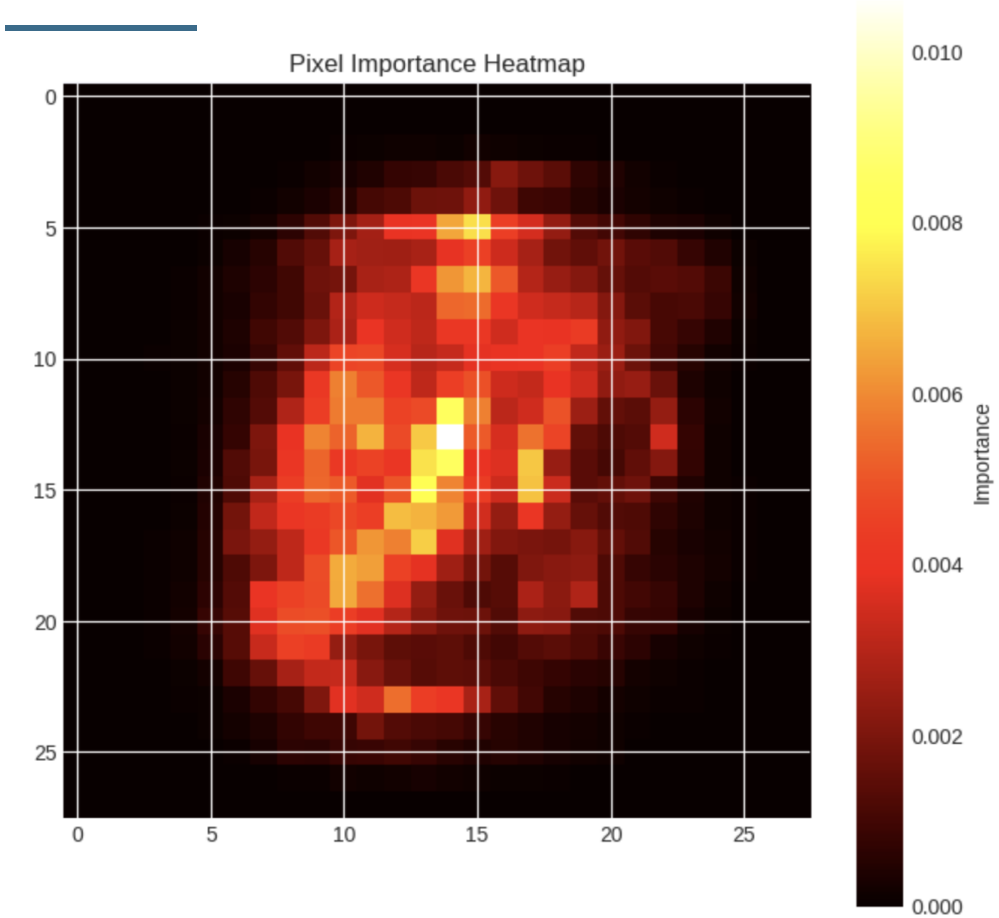


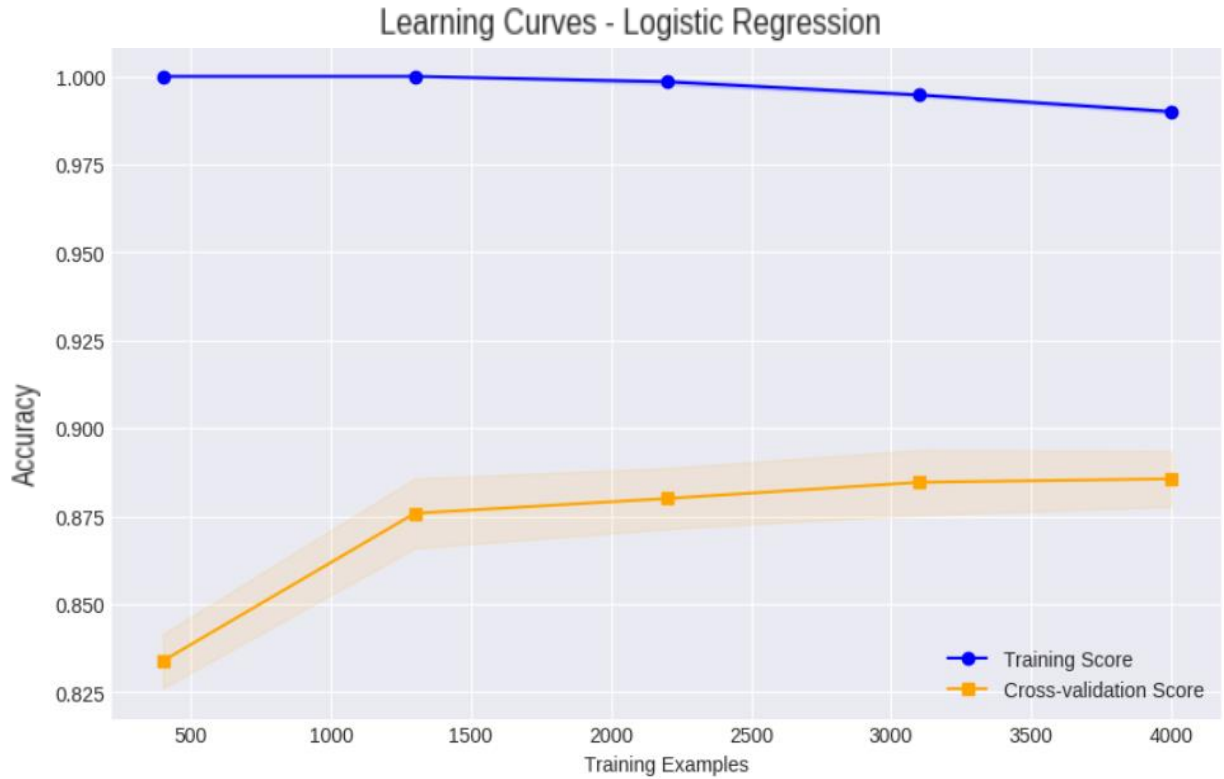
Optimized with Scikit-learn

Confusion Matrix: Scratch (90.42%) → Scikit-learn (92.44%) (+2.02% Gain)



What Logistic Regression Actually Sees





- The model learns that center pixels carry the most discriminative information;
- Edges contribute almost nothing, matching where digits are actually written.

- Training accuracy near 100% but cross-validation plateaus at ~88%;
- the gap indicates some overfitting, but the real bottleneck is the model's linear capacity (high bias).

kNN: Finding Optimal k Through Tuning

Approach

- Classify by majority vote of k nearest neighbors
- Euclidean distance in 784-D pixel space (magnitude meaningful; cosine better for sparse data)
- Tested k = {1, 3, 5, 7, 9, 11, 13, 15}
- **Best model: k=1, Validation Acc:** ~94.8% (during tuning)

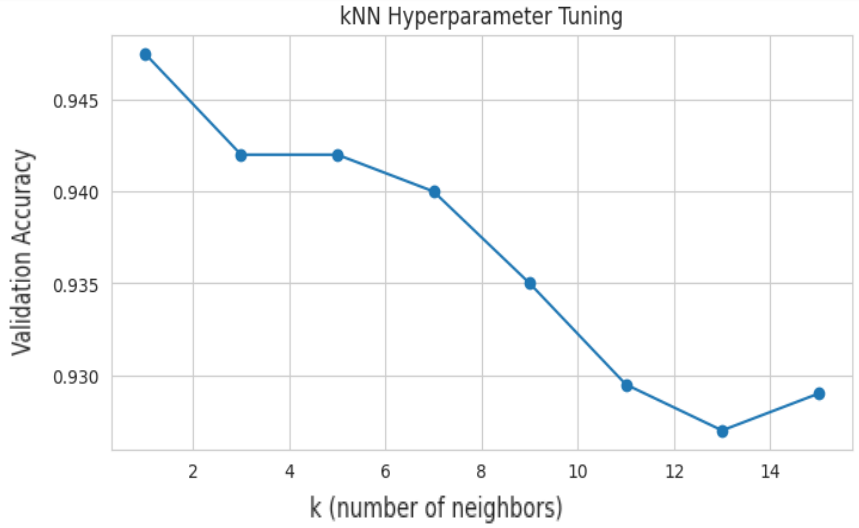
  Final Test Acc: 95.89% (on unseen 10k set)

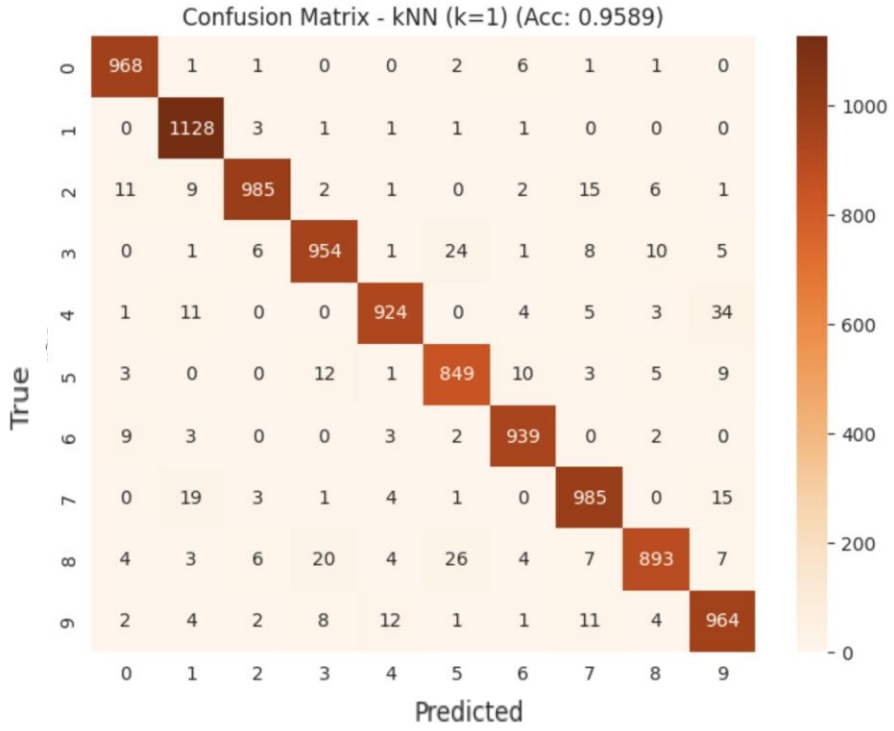
Result

- kNN outperforms logistic regression (by handling nonlinear class boundaries implicitly)
- Computationally expensive at test time, must store and search the entire training set
- Errors on difficult pairs (e.g. 7/9) reduced by ~52% compared to Logistic Regression

Why k=1 Works Here

- MNIST is clean, well-separated data
- On noisier data, higher k generalizes better





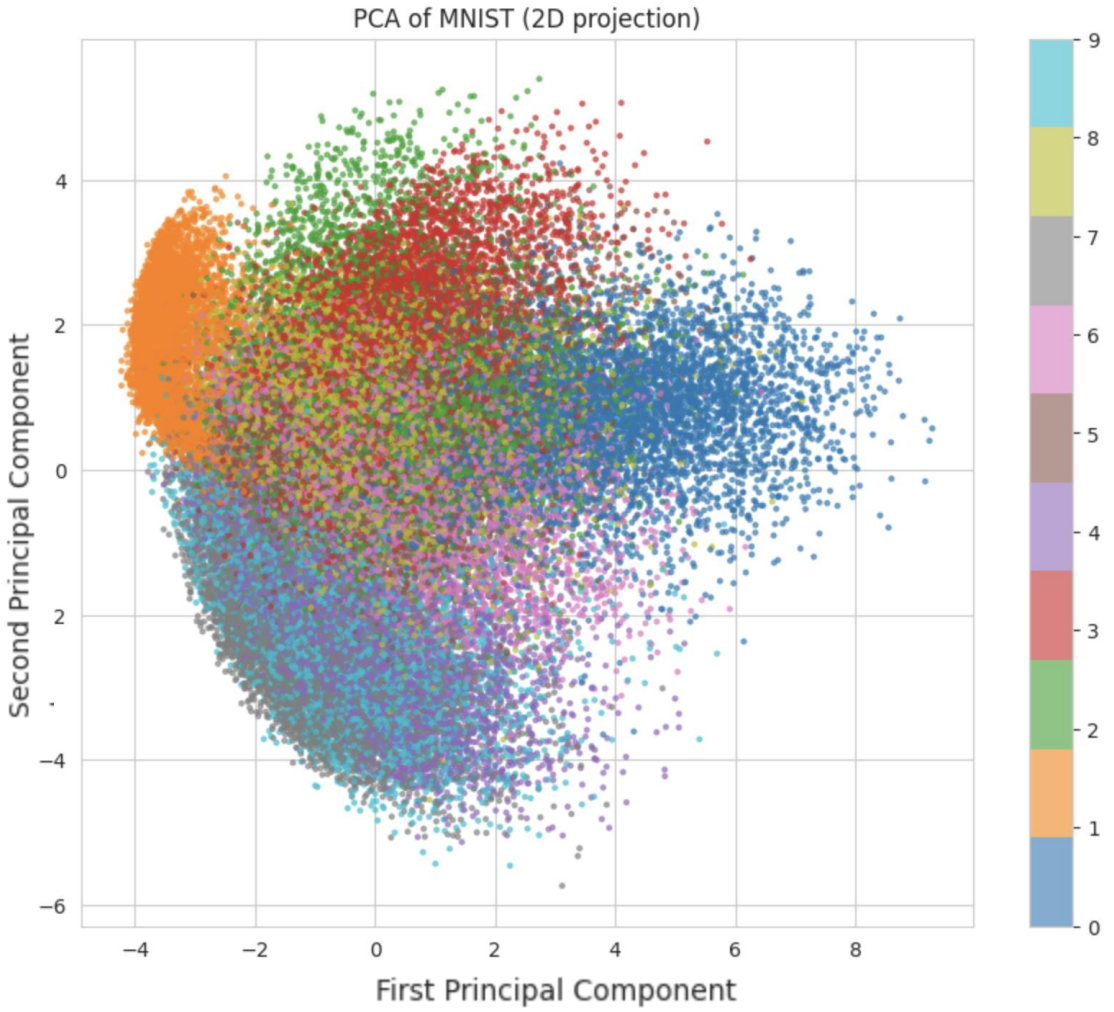
PCA Dimensionality Reduction

Manual PCA Implementation Steps

- Mean-center data
- Compute 784×784 covariance matrix
- Eigenvalue decomposition
- Sort by explained variance
- Verified with sklearn PCA (full dataset)

Insights

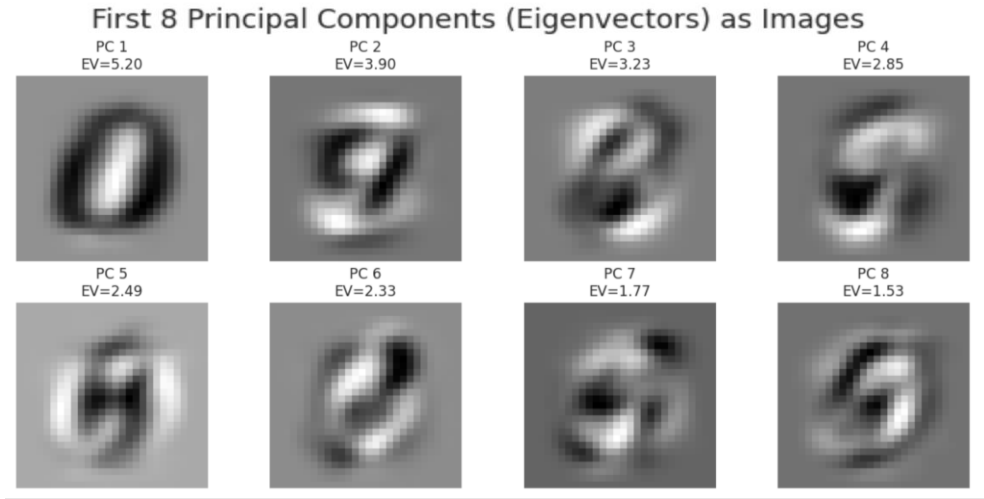
- First 2 Principal Components explain only 17% of variance
- Plot shows significant cluster overlap
- Digits 0 & 1 are relatively separable (large inter-cluster distance)
- Digits 4 & 9 overlap considerably
- Highlights the high-dimensionality of data, requiring more components for meaningful separation

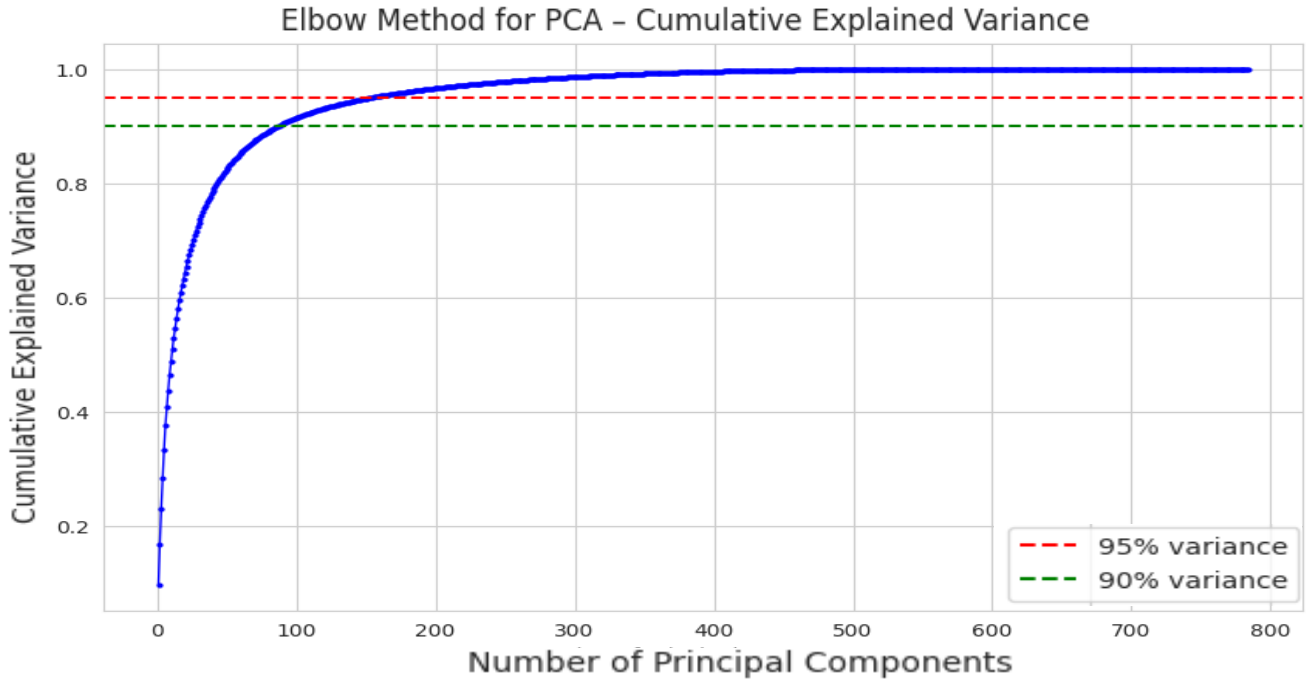


PCA Reveals 150 Components for 95% Variance

Insights

- The “elbow” appears around 50–100 PCs
  - ~100 PCs for 90%
  - ~150 PCs for 95%
- 2D overlap explains why Logistic Regression plateaus at ~92.4%: linear separability is limited
- Models trained on ~150-D space perform similarly to those using all 784 features
- PCA maximizes variance, not necessarily class separability, as seen in the 2D scatter plot
  - **Solution: Non-linear methods such as t-SNE better preserve class separability.**



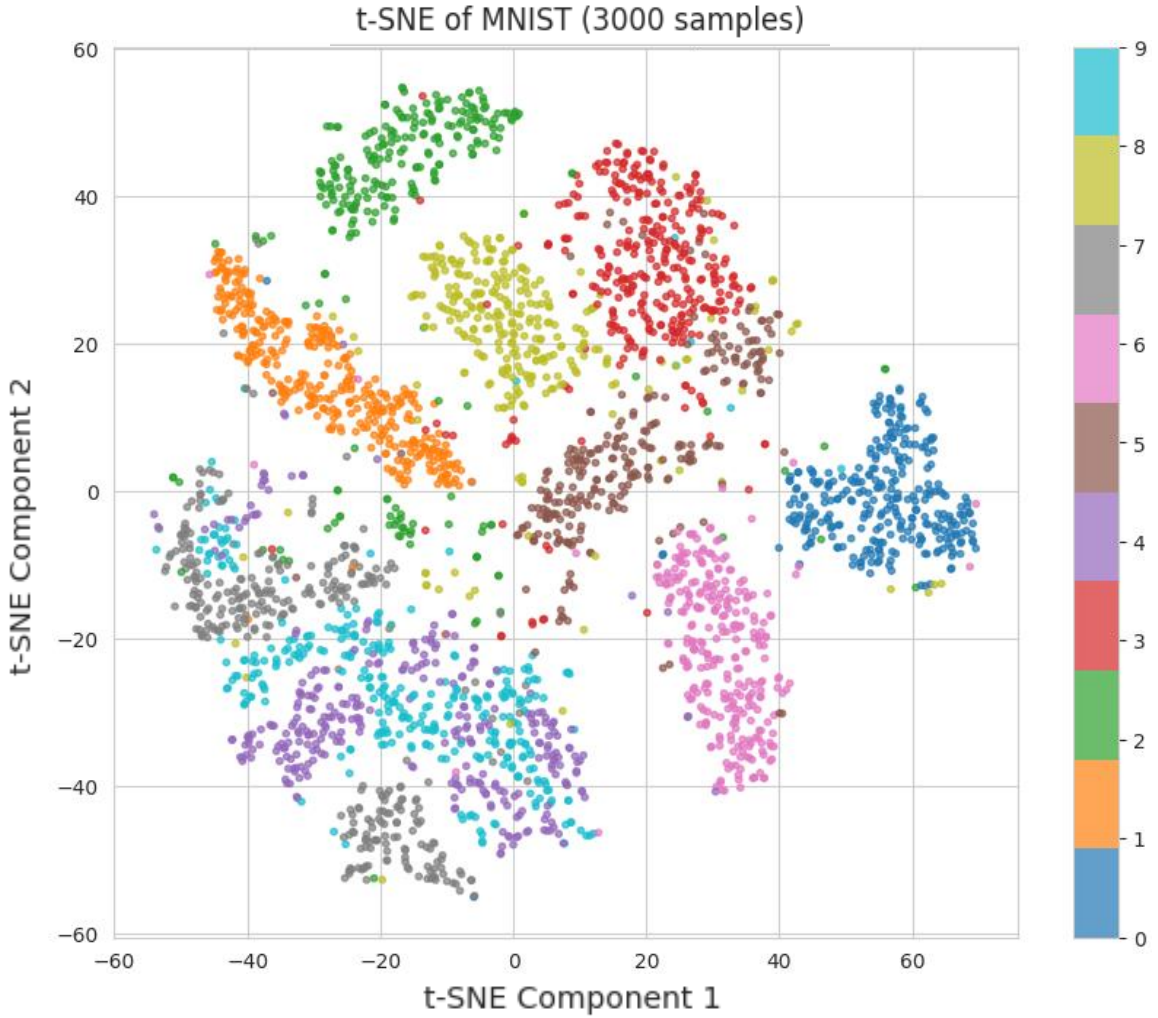


t-SNE Reveals Class Structure

- t-SNE preserves local structure, excellent for cluster visualization
- Caveat: Inter-cluster distances are not directly interpretable

Insights

- 10 distinct, well-separated clusters
- Similar digits (e.g. 4/9, 3/5) show overlapping regions
- **Suggests the data lies on a low-dimensional manifold with clear class structure.**
- Confirms why nonlinear models outperform linear classifiers



Parameters:

- perplexity = 30
- n_iter = 1,000
- 3,000 sample subset due to computational cost

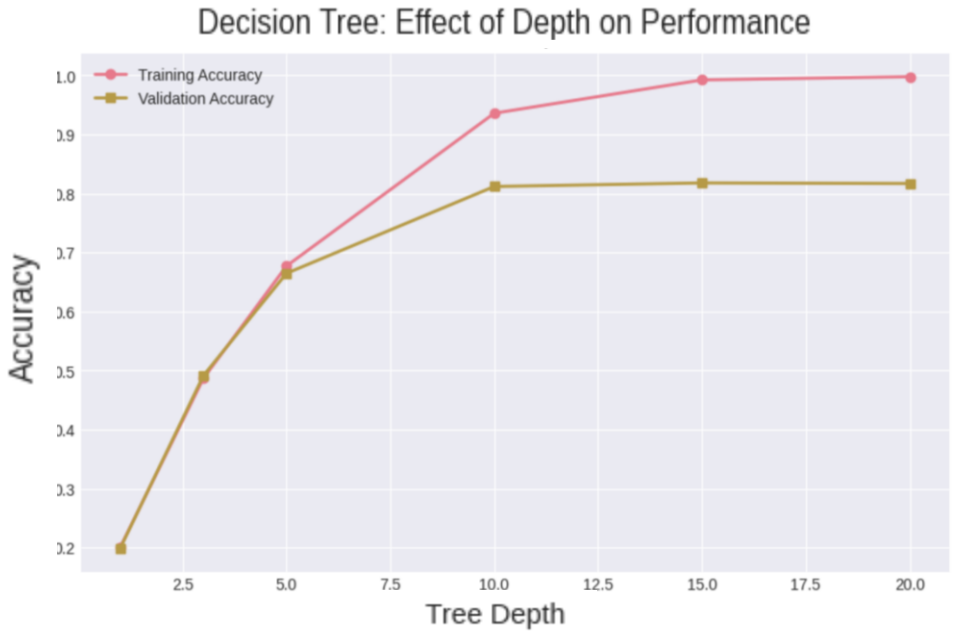
From Overfitting Trees to Random Forest Ensembles

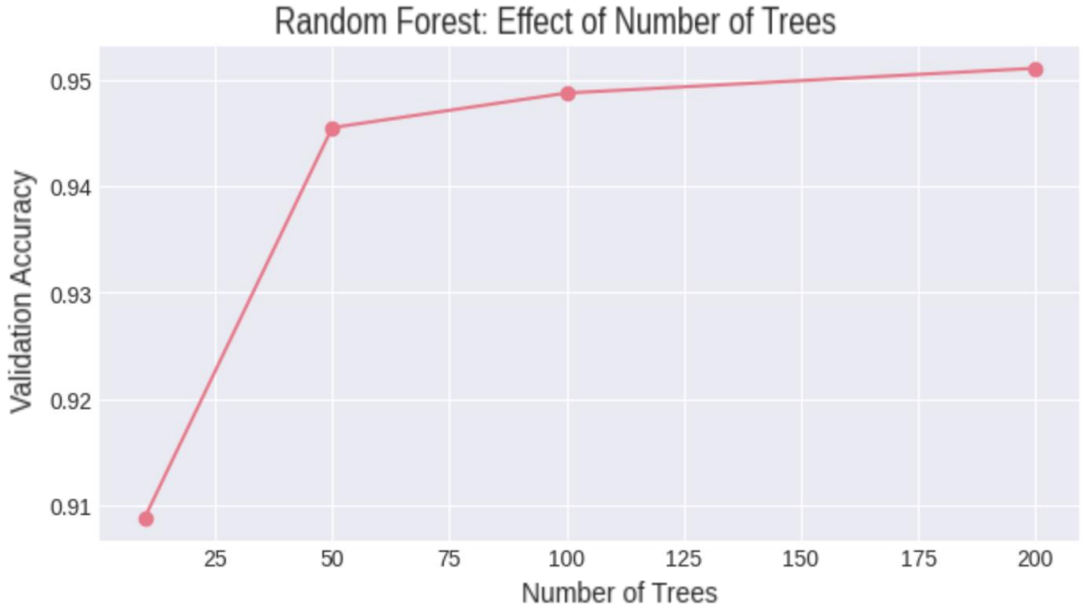
Decision Tree Overfitting

- Training accuracy: 100% (memorizes data)
- Validation peaks at depth 10–15
- Cost-complexity pruning: optimal $\alpha = 0.000235$

Random Forest Ensemble

- Ensemble of 200 trees
- Improves validation accuracy by ~13% over a single tree by significantly reducing model variance.
- **Bootstrap + feature subsampling = decorrelates individual trees**





Insights

- Random Forest: Validation accuracy 95.11% → Final test accuracy 96.94% (using full 60K training set for final model)
- This significantly outperforms a single decision tree (86.61% Test Accuracy) by reducing model variance via ensemble averaging

Why Single Trees Fail: Motivating Ensembles

	Decision Tree	Random Forest (Tuned)
Splits on	Individual pixel brightness	Majority vote of 200 decorrelated trees
Accuracy	86.61%	96.94%
Overfitting	Severe (memorises 100% training)	Controlled
Interpretable	High	Low
Speed	Fast	Slower
Key Parameter	Tested depth 1-20 / Optimal: 10-15	Tested 25-200 trees / Best tested: 200

By averaging 200 trees trained on random subsets, Random Forest significantly reduces the variance that causes single trees to overfit.

Conclusion: The ensemble approach effectively trades interpretability for a 10.33% gain in accuracy, making it our primary non-deep-learning baseline.

Key Takeaways for Traditional ML

1. Model Complexity Matters:

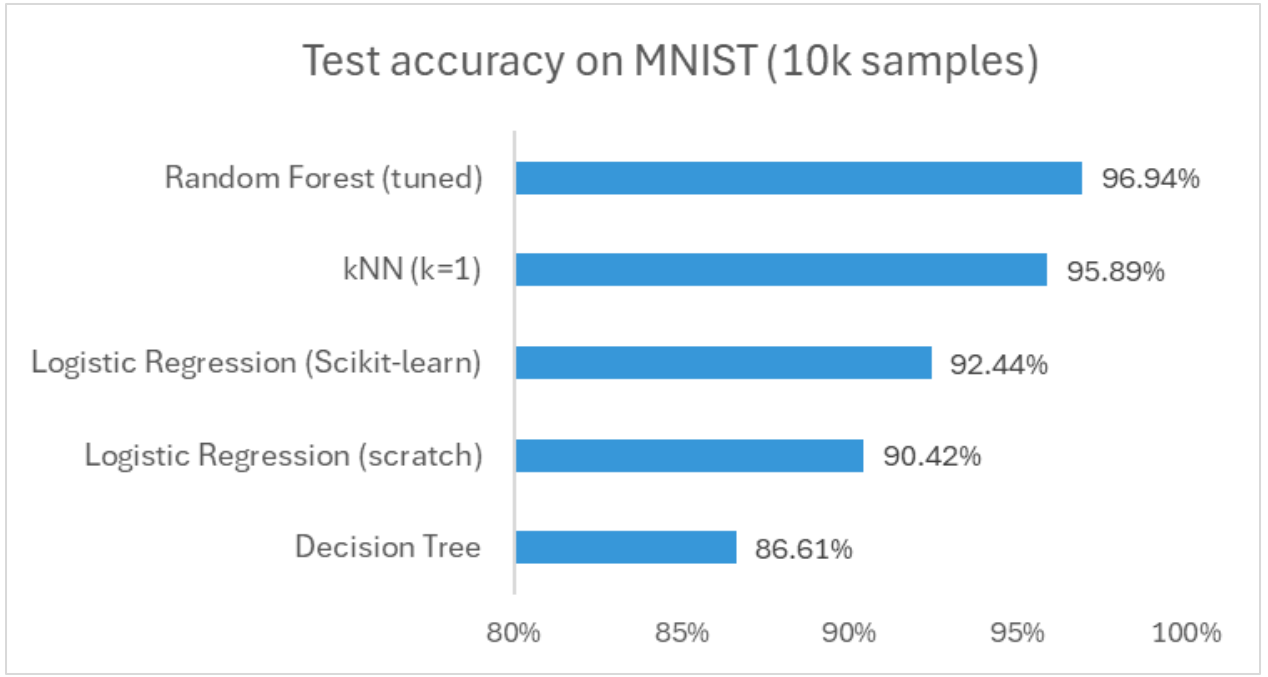
More complex models (e.g. Random Forest) capture non-linear patterns better than linear models.

2. Ensemble Methods Work:

Random Forest significantly outperforms single decision tree by reducing variance.

3. Hyperparameter Tuning is Essential:

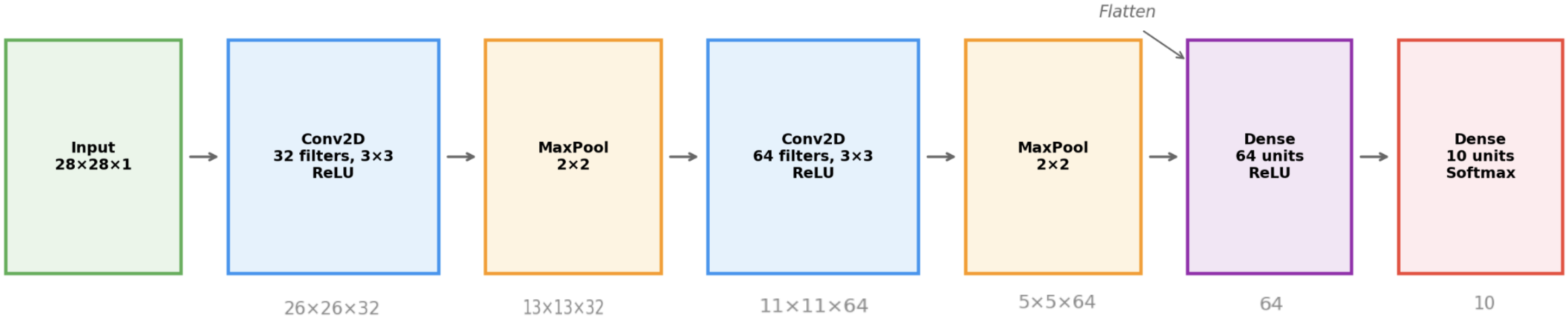
Tuning lifted Logistic Regression from 90.42% (scratch) to 92.44% (Scikit-learn), but it still hits a ceiling; linearity is the bottleneck, not hyperparameters.



Practical Recommendations:

- **Quick prototyping**: Tuned Logistic Regression — fast but limited by linearity
- **Best accuracy:** Random Forest — accurate but black-box
- **Minimal setup:** kNN — no training, but slow at prediction
- **Interpretability:** Pruned Decision Tree — explainable but lower accuracy

CNN Achieves 99.08% Test Accuracy



Deep Learning Results

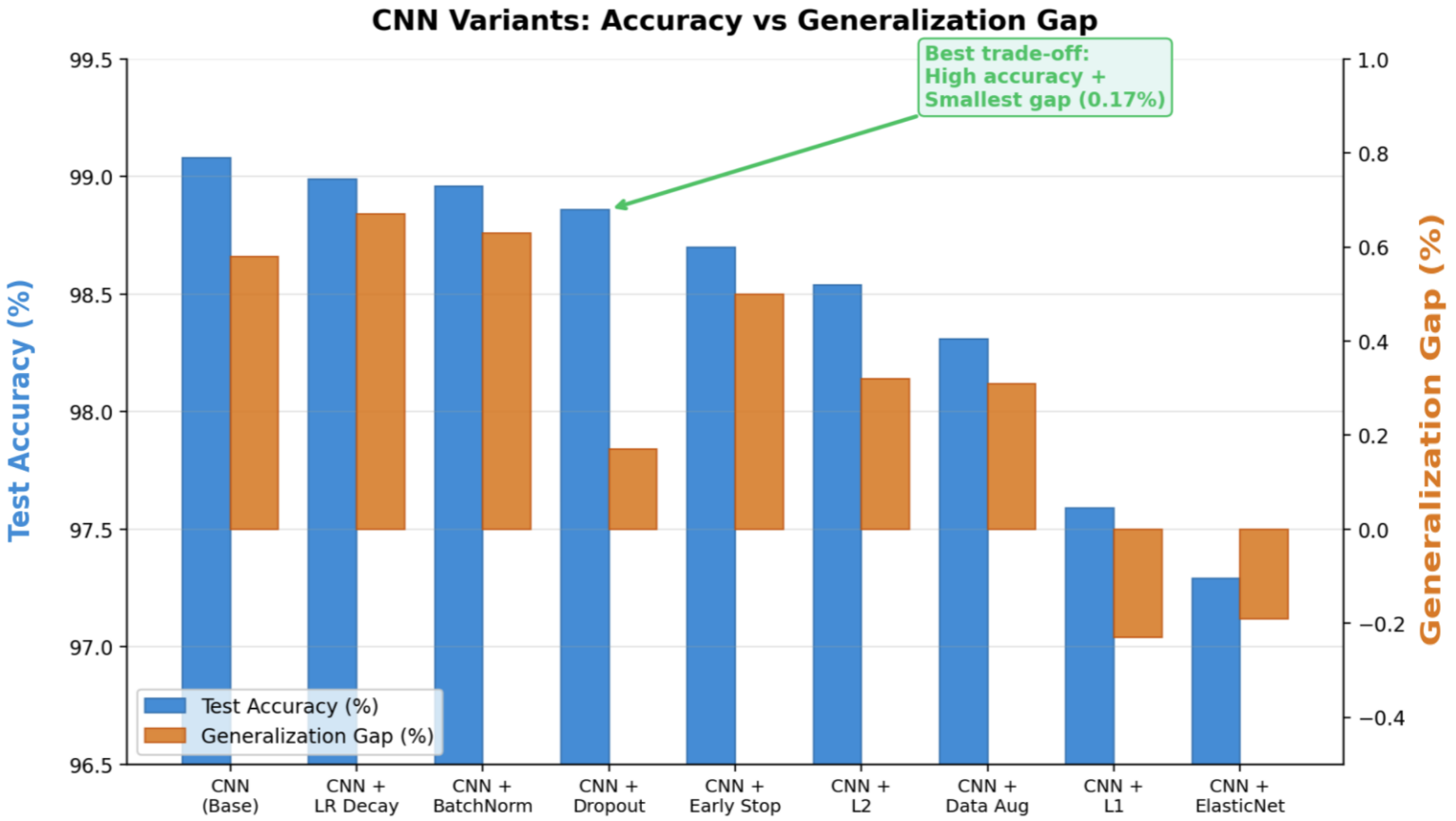
- CNN architecture: 2x (Conv2D → MaxPool) → Dense → Softmax
- **Training:** Adam optimizer | Categorical Cross-Entropy | Batch 32 | 10 Epochs | 20% validation split
- Unlike flattened models, CNNs preserve spatial structure — neighboring pixels matter.
- **+2.14% over best traditional ML (Random Forest at 96.94%)**

CNN Regularization: Less is More

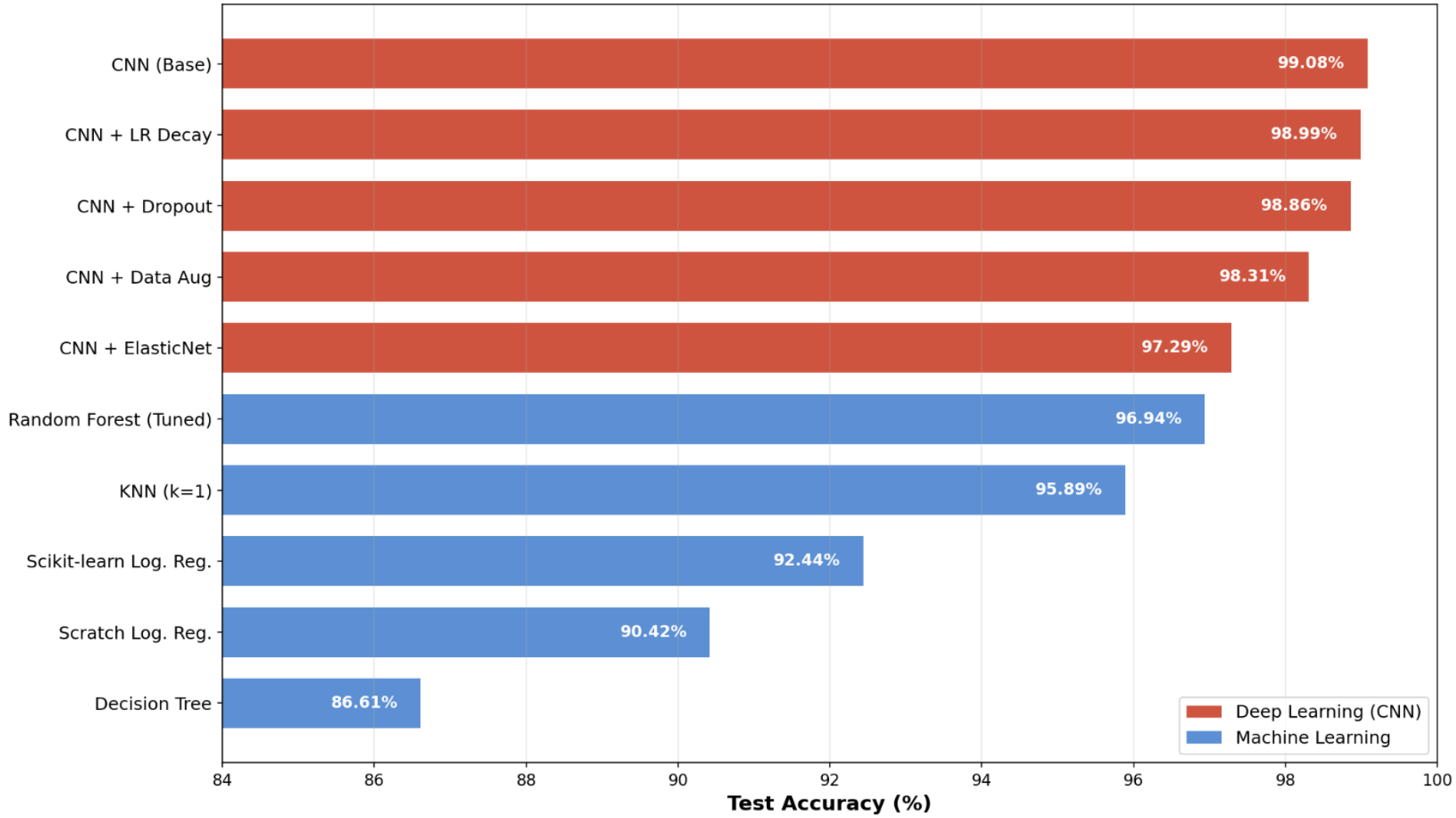
- Regularization Explored (9 variants tested)
- Dropout (0.5): Best trade-off — 98.86% accuracy with only 0.17% generalization gap
- BatchNorm, L2, L1, ElasticNet, Early Stopping, LR Decay, Data Augmentation

Key finding:

The strong base CNN means aggressive regularization (L1, ElasticNet) hurts more than it helps.



From 86.61% to 99.08%: Complete Model Comparison



Even the weakest CNN variant outperforms the best traditional ML model

Model Comparison: Strength and Weaknesses

Model	Test Accuracy	Training Time	Interpretability
k-NN (k=1)	95.89%	None / Slow (test)	High (local) / Low (Global)
Logistic Regression	92.44%	Very fast	**High (pixel-wise weights)**
Random Forest	96.94%	Moderate	Low
CNN	**99.08%**	Slow (GPU recommended)	Very Low (black box)

While the CNN is our accuracy "winner", Logistic Regression is the winner for transparency – it allows us to see exactly which pixels drive a classification decision.

Conclusions

1. Architecture over Tuning:

CNNs achieve 99.08% by learning spatial features; Random Forest reaches 96.94% with no GPU — model choice depends on accuracy, computational cost, and interpretability.

2. Manifold Discovery:

PCA reveals high intrinsic dimensionality (150 components for 95% variance); t-SNE reveals clear class structure — explaining why nonlinear models dominate.

3. Regularization Strategy:

Dropout is the most effective regularization (0.17% generalization gap); aggressive methods like L1/ElasticNet over-constrain the model on relatively simple datasets like MNIST.

Thank You

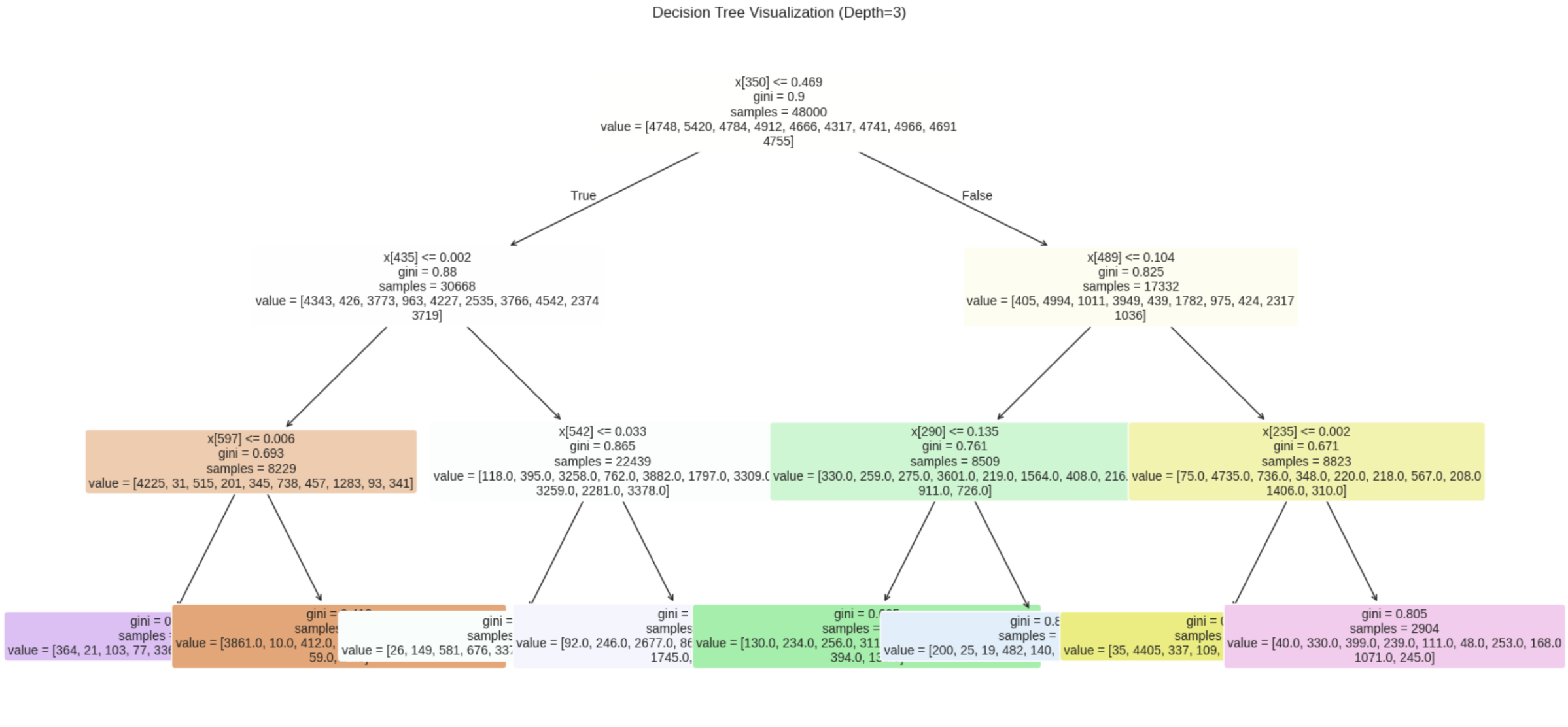
AI Declaration

We used Anthropic's Claude (Sonnet and Opus 4.6) and Google's Gemini 3.1 Pro AI tools contributed to testing ideas, improving clarity, and refining drafts. All outputs from AI were critically evaluated and integrated by the people in the group to ensure accuracy, and alignment with academic standards.

We acknowledge that any draft by AI tools was refined through multiple rounds of human editing. The group retains full responsibility for all substantive content, including argument development, data analysis, and conclusions. This declaration affirms transparency in AI usage while maintaining human scholarly ownership and accountability. It fully complies with NUS's expectations to avoid plagiarism, misrepresentation, and academic misconduct in the age of AI-assisted work to the best of our knowledge. The group makes this declaration in good faith, acknowledging potential imperfections despite rigorous efforts to maintain academic integrity.

Appendix

Decision Tree Visualization



Why Hyperparameter Tuning?

Model

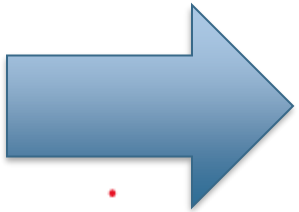
Softmax logistic regression for 10 MNIST classes

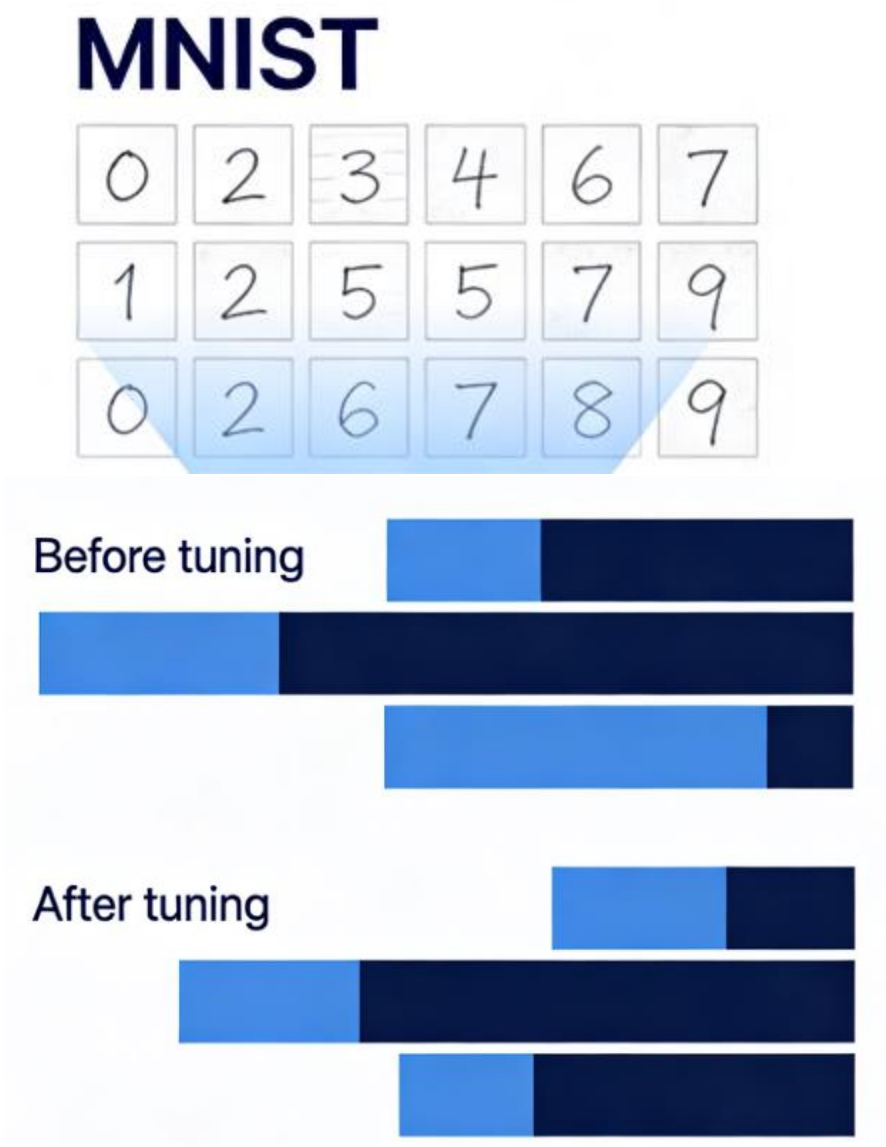
Tuning

Search C, penalty, solver, max_iter

Impact

Reduces overfitting, improves test accuracy, stabilizes convergence





- The model is trained by minimizing cross-entropy loss with a regularization term:
- L1 regularization encourages sparsity.
- L2 regularization keeps weights small and stable.

Classical Model Performance

	Model	Test Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)
0	Random Forest (Tuned)	0.9694	0.969271	0.969110	0.969172
1	SVM (RBF, Tuned)	0.9688	0.968710	0.968404	0.968517
2	K-Nearest Neighbour	0.9589	0.959000	0.958391	0.958518
3	Tuned Logistic Regression	0.9245	0.923627	0.923353	0.923355
4	Default Sklearn Logistic Regression	0.9244	0.923501	0.923269	0.923273
5	Scratch Logistic Regression	0.9042	0.904329	0.902619	0.902666
6	Decision Tree	0.8661	0.864438	0.864082	0.864059

Unsupervised Clustering Performance:

	Model	ARI	Silhouette	Homogeneity	Completeness	V-measure
0	K-Means Clustering	0.359932	0.060898	0.490645	0.501986	0.496251

Deep Learning Improvisation Techniques with results

	train_acc	test_acc	precision	recall	f1	gen_gap
CNN (base)	0.9966	0.9908	0.9908	0.9907	0.9907	0.0058
CNN + LR Decay	0.9966	0.9899	0.9898	0.9897	0.9898	0.0067
CNN + BatchNorm	0.9959	0.9896	0.9897	0.9895	0.9895	0.0063
CNN + Dropout	0.9903	0.9886	0.9886	0.9885	0.9885	0.0017
CNN + Early Stopping	0.9920	0.9870	0.9871	0.9869	0.9870	0.0050
CNN + L2	0.9886	0.9854	0.9855	0.9852	0.9852	0.0032
CNN + Data Augmentation	0.9862	0.9831	0.9833	0.9830	0.9830	0.0031
Baseline Dense	0.9902	0.9765	0.9767	0.9764	0.9764	0.0137
CNN + L1	0.9736	0.9759	0.9758	0.9758	0.9757	-0.0023
CNN + ElasticNet	0.9710	0.9729	0.9729	0.9727	0.9727	-0.0019
CNN + Autoencoder Features	0.8982	0.9073	0.9117	0.9063	0.9061	-0.0091

CNN Training

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_5 (Dense)	(None, 64)	102,464
dense_6 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

Model Architecture

- Input: 28×28 grayscale image
- Conv (32 filters, 3×3, ReLU) → MaxPooling
- Conv (64 filters, 3×3, ReLU) → MaxPooling
- Flatten → Dense (64, ReLU)
- Output: 10 classes (Softmax)

Training Configuration

- Optimizer: Adam
- Loss: Categorical Cross-Entropy
- Batch Size: 32
- Epochs: 10
- Validation Split: 20%

Learning Process

- Forward pass → predictions
- Compute loss
- Backpropagation → update weights
- Repeat across epochs